Is PPO the best policy?

The deployed PPO policy against tabular Q-learning, deep Q-learning (DQN), a hand-written controller and two planners, on held-out layouts in a 2D simulator and in Gazebo.

90.2%
PPO success, held-out test with safety layer

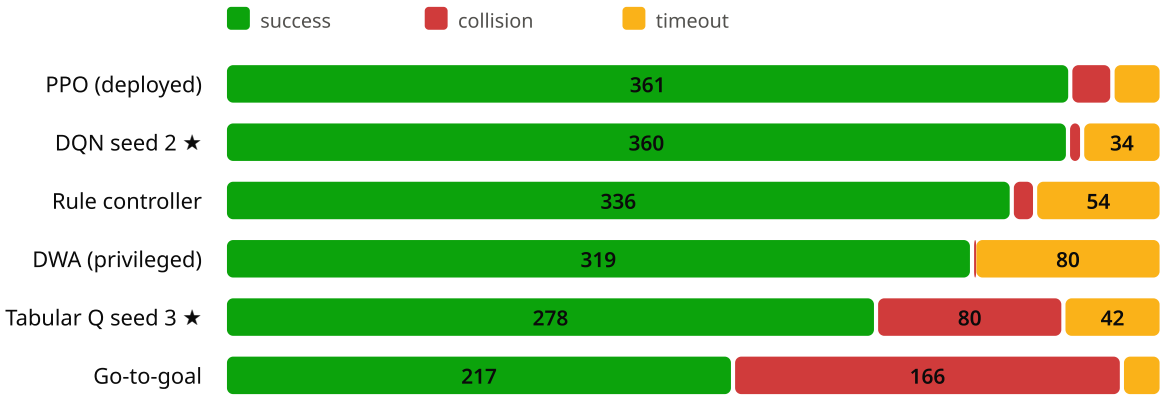
90.0%
DQN (chosen seed): statistically level with PPO, 3× fewer collisions

84.5%
Rule controller without safety layer
vs PPO 68.0% without it

Verdict

With the safety layer (as deployed), PPO is matched, not beaten: DQN ties it in 2D (90.0% vs 90.2%) and in Gazebo (73 of 80 robots each), with fewer collisions but slower arrivals. **Without the safety layer, PPO is weak:** it learned to lean on that layer, and a classic dynamic-window planner using the same sensors beats it by 16.5 points in 2D. That planner does not hold up in Gazebo (80%), where both learned policies stay at 91%. Tabular Q-learning trails far behind (69.5%).

At a glance: what happened to the 400 test robots (2D, safety layer on)



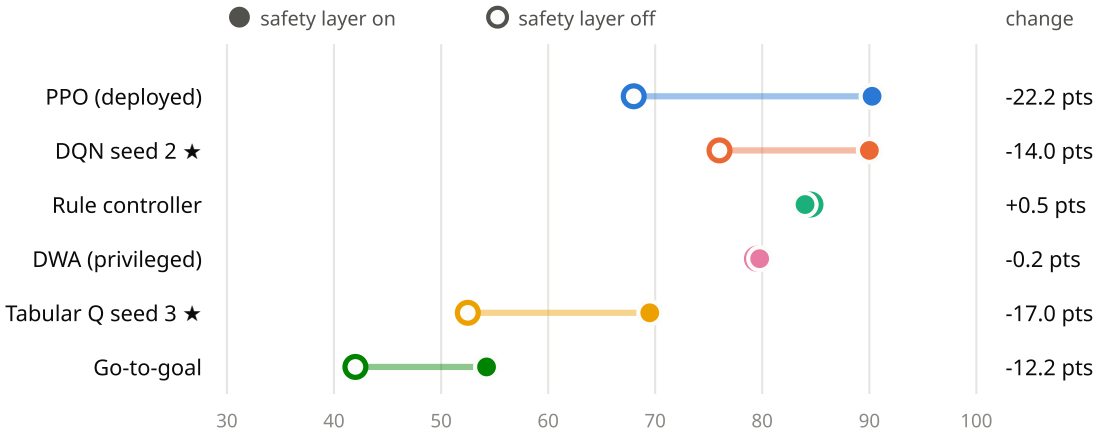
Branch `benchmark-policies`, tag `benchmark-v1` · 10 November 2026 · laptop (i5-12450H) + Google Colab (Tesla T4) · every number regenerated from raw per-robot outcomes

1 Key findings

1. **PPO and DQN are tied in the deployed configuration.** Test success 90.2% vs 90.0% (difference −0.2 points, 95% interval −4.0 to +3.5). Collisions 4.5% vs 1.5%. DQN is slower to the goal (16.1 s vs 10.9 s) and times out more (8.5% vs 5.2%). DQN trained for only 75% of PPO's budget and with far less tuning.
2. **DQN's seeds disagree**: 88.0%, 90.0%, 83.0%. PPO exists as a single trained policy, so its own seed spread is unknown; its 90.2% may be a favourable draw just as DQN's best seed is.
3. **PPO depends on the safety layer.** The layer replaces 26% of PPO's actions; without it PPO falls to 68.0% and a quarter of its robots hit another robot. DQN falls to 76.0%. The rule controller does not change (84.0% → 84.5%).
4. **The hand-written controller wins in 2D without the safety layer** (+16.5 points over PPO, interval +9.5 to +23.2), with 0.5% collisions, but **drops to 80.0% in Gazebo** (−11.2 points vs PPO), mainly from timeouts. Learned policies transferred better to realistic physics and sensing.
5. **Tabular Q-learning is limited by its state compression**: 61–70% across seeds, ~20% collisions. A 24× finer table learned more slowly and ended worse (54.2%).
6. **Planners that see the true map are not enough either**: DWA reaches 79.8% with almost no collisions, but 20% of its robots time out.

Success on the held-out test layouts, safety layer on and off

2D simulator, layouts 100–199 (400 robot runs per method). Filled dot: safety layer on (deployed); hollow: off.



2 What was compared, and how

The task

Four differential-drive robots each drive to their own goal in a random arena of obstacles, avoiding obstacles, walls and each other. Every method controls each robot from the **same 59 numbers** (36 lidar sectors, goal distance and direction, own speed, camera estimates of the three nearest robots) and picks one of the **same 45 commands** (speed × turn rate), ten times a second. A rule-based **safety layer** can veto a command that leads into another robot; it is part of the deployed system.

Data split

Test: layouts 100–199, never used to train, tune or pick anything. (PPO's checkpoint was picked on layouts 0–99, so those are shown only for reference.) New methods trained on layouts 1000–2899 and were tuned and checkpoint-selected on **validation layouts 2900–2999** only.

Statistics

Success, collision and timeout per robot. 95% intervals by bootstrap over layouts (10,000 resamples), because the four robots of one layout are correlated; differences to PPO paired on the same layouts.

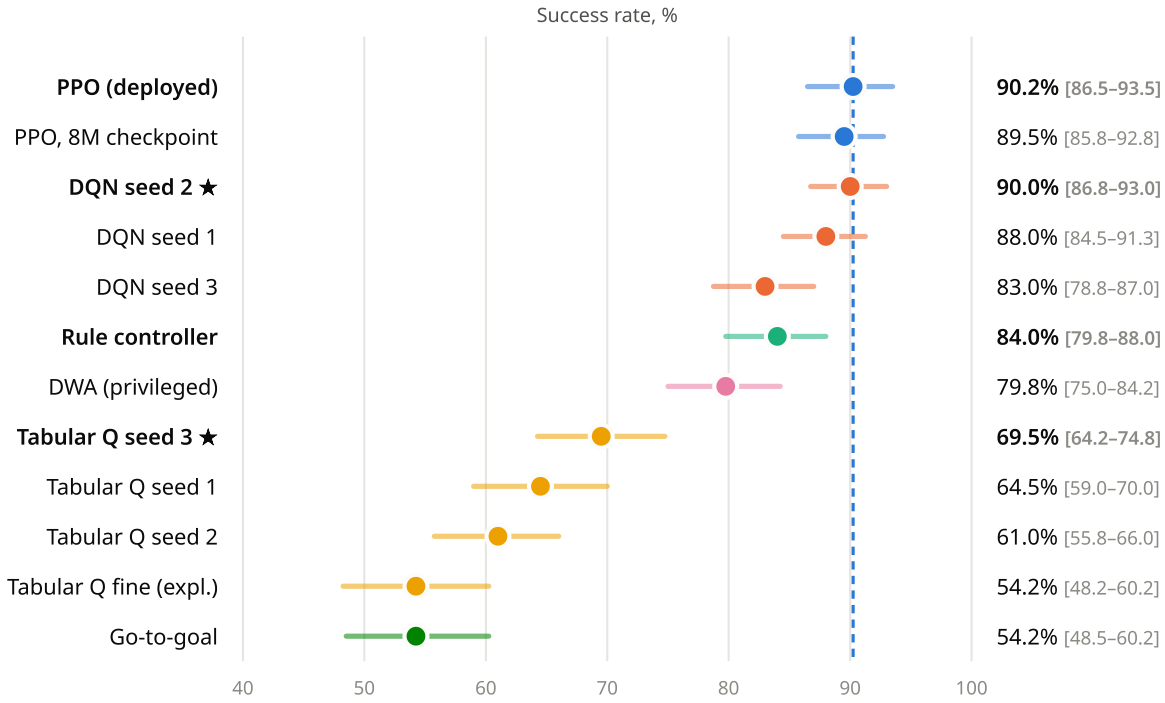
Contenders

Method	What it is	Training here
PPO	Deployed policy (existing), MLP 59-256-256-45	none; 10 earlier runs
DQN	Deep Q-learning, same network shape	6 configs × 1M; 3 seeds × 6M
Rule	Dynamic-window search: predicts each command's arc, vetoes collisions, scores progress	50 parameter sets
Tabular Q	Q-table over 6,480 discretised states	9 configs; 3 seeds × 8M
DWA	Planner that sees the **true map** (privileged)	none
Go-to-goal	Drives straight at the goal	none

Written before any result. The protocol (contenders, split, budgets, selection rule “success − 0.5 × collision” on validation) was committed first. Later amendments, each recorded with its reason, are in section 8.

3 2D results with the safety layer (deployed configuration)

Success rate with 95% interval; dashed line: PPO. ★ = seed chosen on validation (never on test).

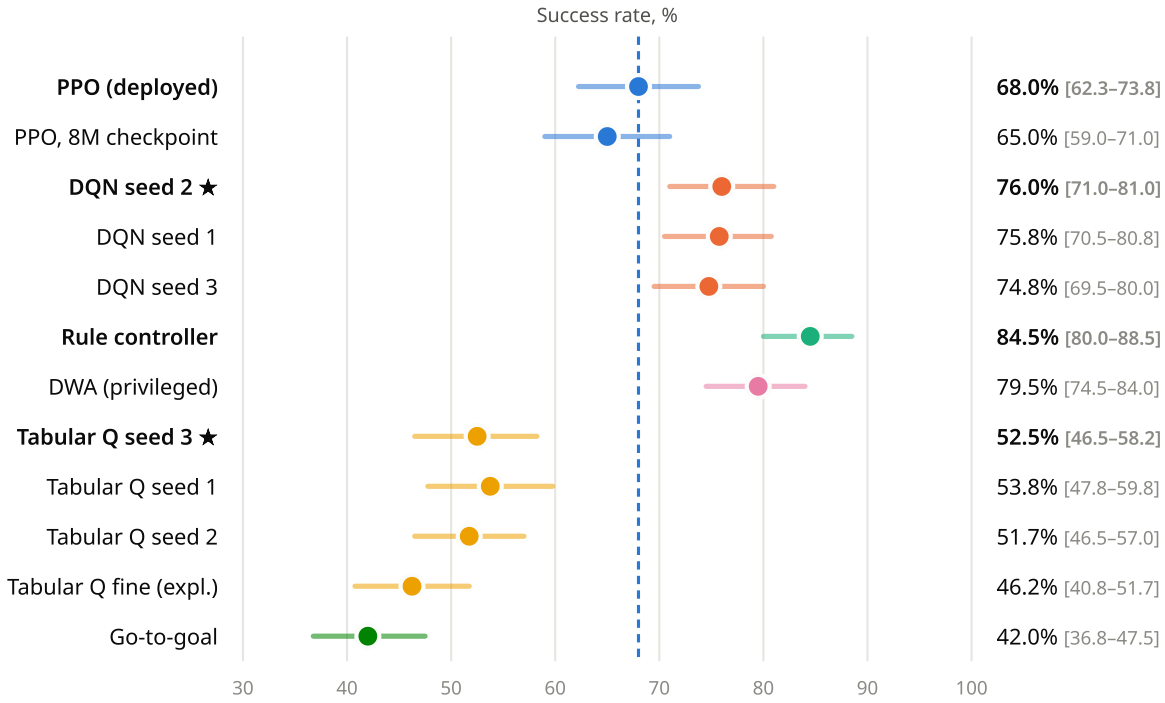


Method	Success % [95% CI]	Collision %	Timeout %	All four %	Time to goal s	Δ success vs PPO [95% CI]
PPO (deployed)	90.2 [86.5, 93.5]	4.5	5.2	71	10.9	—
PPO, 8M checkpoint	89.5 [85.8, 92.8]	4.0	6.5	69	10.7	-0.8 [-3.0, +1.8]
DQN seed 2 ★	90.0 [86.8, 93.0]	1.5	8.5	68	16.1	-0.2 [-4.0, +3.5]
DQN seed 1	88.0 [84.5, 91.3]	0.5	11.5	64	15.4	-2.2 [-6.0, +1.5]
DQN seed 3	83.0 [78.8, 87.0]	0.5	16.5	51	15.4	-7.2 [-11.0, -3.5]
Rule controller	84.0 [79.8, 88.0]	2.5	13.5	56	9.6	-6.2 [-10.2, -2.2]
DWA (privileged)	79.8 [75.0, 84.2]	0.2	20.0	47	11.9	-10.5 [-15.0, -6.0]
Tabular Q seed 3 ★	69.5 [64.2, 74.8]	20.0	10.5	31	15.9	-20.8 [-25.5, -16.0]
Tabular Q seed 1	64.5 [59.0, 70.0]	20.2	15.2	22	14.7	-25.8 [-31.0, -20.2]
Tabular Q seed 2	61.0 [55.8, 66.0]	27.5	11.5	17	13.8	-29.2 [-34.2, -24.5]
Tabular Q fine (expl.)	54.2 [48.2, 60.2]	26.0	19.8	18	16.4	-36.0 [-41.8, -30.2]
Go-to-goal	54.2 [48.5, 60.2]	41.5	4.2	20	8.9	-36.0 [-41.5, -30.5]

Override = share of robot-steps where the safety layer replaced the method's command: PPO 25.9%, DQN 8.0%, rule 9.8%.
Selecting PPO's checkpoint on layouts 0–99 made little difference: the never-selected 8M checkpoint scores 89.5%.

4 Without the safety layer

Same layouts and methods, safety layer removed. Dashed line: PPO without the layer.



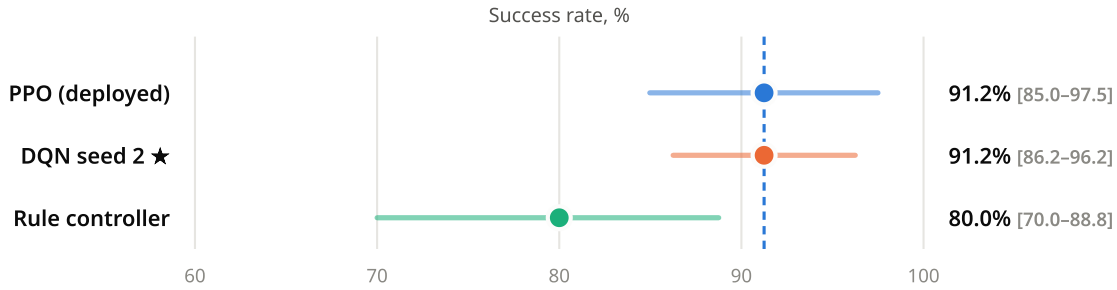
Method	Success % [95% CI]	Collision % (robot)	Timeout %	All four %	Time to goal s	Δ vs PPO, both off
PPO (deployed)	68.0 [62.3, 73.8]	29.2 (25.2)	2.8	39	10.3	—
PPO, 8M checkpoint	65.0 [59.0, 71.0]	32.5 (29.2)	2.5	36	10.0	-3.0 [-7.0, +0.8]
DQN seed 2 ★	76.0 [71.0, 81.0]	16.5 (15.8)	7.5	47	15.7	+8.0 [+1.8, +14.2]
DQN seed 1	75.8 [70.5, 80.8]	12.5 (12.5)	11.8	47	14.6	+7.8 [+1.5, +13.8]
DQN seed 3	74.8 [69.5, 80.0]	12.5 (12.5)	12.8	43	15.3	+6.8 [-0.8, +14.2]
Rule controller	84.5 [80.0, 88.5]	0.5 (0.0)	15.0	57	9.6	+16.5 [+9.5, +23.2]
DWA (privileged)	79.5 [74.5, 84.0]	0.0 (0.0)	20.5	48	11.9	+11.5 [+4.2, +18.5]
Tabular Q seed 3 ★	52.5 [46.5, 58.2]	41.8 (23.5)	5.8	17	13.8	-15.5 [-23.0, -8.0]
Tabular Q seed 1	53.8 [47.8, 59.8]	31.8 (14.0)	14.5	15	13.8	-14.2 [-21.2, -7.2]
Tabular Q seed 2	51.7 [46.5, 57.0]	38.2 (16.8)	10.0	10	13.0	-16.2 [-23.8, -9.0]
Tabular Q fine (expl.)	46.2 [40.8, 51.7]	39.0 (16.2)	14.8	7	15.2	-21.8 [-29.2, -14.5]
Go-to-goal	42.0 [36.8, 47.5]	58.0 (19.0)	0.0	9	8.1	-26.0 [-32.2, -19.8]

Why the order flips. PPO and DQN were trained with the safety layer always on, so they learned to let it resolve robot-robot conflicts: without it, most of their collisions are with other robots (PPO 25.2 of 29.2 points, DQN 15.8 of 16.5). The rule controller does its own collision check every step, so removing the external layer changes nothing for it.

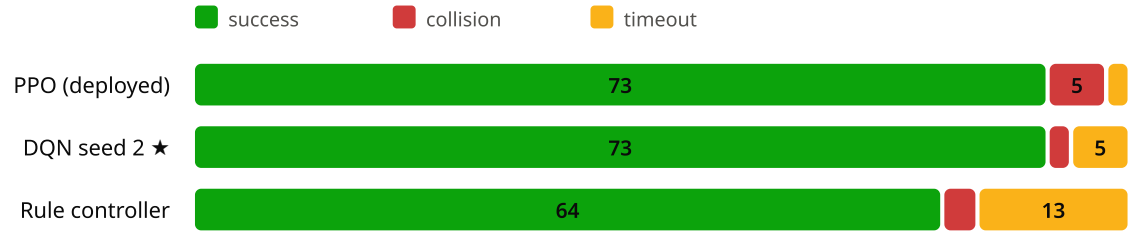
5 Gazebo confirmation

Full pipeline: overhead camera with AprilTags, 360-beam lidar, odometry and physics; layouts 100–119 (80 robots), safety layer on. PPO, the validation-chosen DQN seed and the better of rule / tabular Q from the 2D test.

Success rate with 95% interval (bootstrap over the 20 layouts)



Outcome of each of the 80 robots



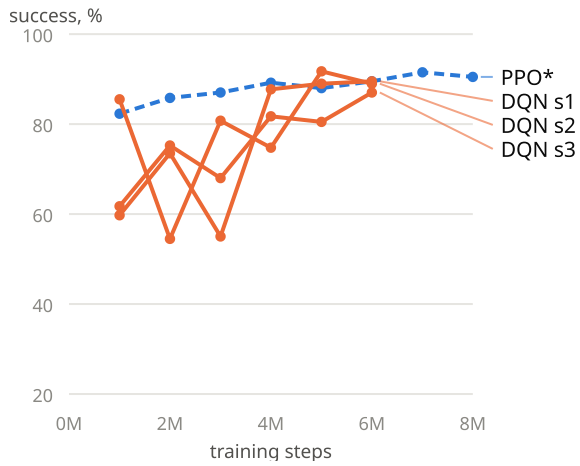
Method	Success % [95% CI]	ok / collision / timeout	All four %	Time to goal s	Δ vs PPO [95% CI]
PPO (deployed)	91.2 [85.0, 97.5]	73 / 5 / 2	70	10.4	—
DQN seed 2 ★	91.2 [86.2, 96.2]	73 / 2 / 5	65	15.5	+0.0 [-5.0, +5.0]
Rule controller	80.0 [70.0, 88.8]	64 / 3 / 13	45	9.4	-11.2 [-23.8, +0.0]

PPO's 91.2% repeats its earlier Gazebo record (89.8% on layouts 1–50), a check that the benchmark harness drives Gazebo correctly. With 20 layouts the intervals are wide: only the rule controller's drop is close to resolved (upper bound 0.0). Gazebo ends an episode after 70 s (2D: 90 s), which turns slow robots into timeouts.

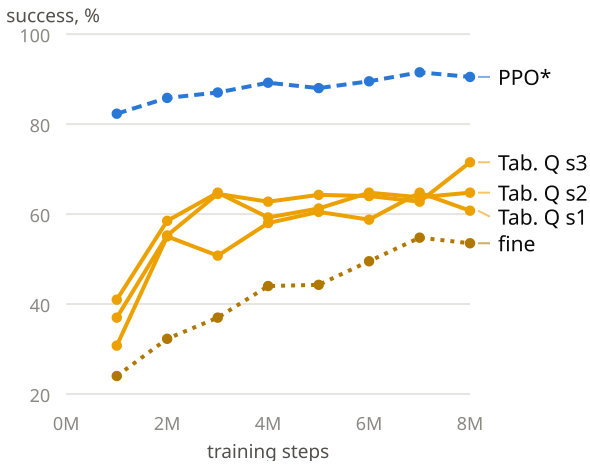
6 How they learn and how they drive

Validation success (layouts 2900–2999) during training, checkpoints every 1M steps. PPO's curve is from its original training log (*measured on layouts 0–99), so it is indicative only. DQN stopped at 6M (Colab reset). DQN reaches PPO's level by 4–5M steps but swings between checkpoints; tabular Q flattens at about 60–70%.

DQN, 3 seeds



Tabular Q, 3 seeds + fine table



Driving style (2D test, safety layer on)

Method	Path efficiency	Mean speed m/s	Time stopped (successful robots)	Steering flips / min	Closest obstacle m (median)	Closest robot m (median)	Safety-layer overrides
PPO (deployed)	0.97	0.127	4.4%	218	0.166	0.150	25.9%
DQN seed 2 ★	0.96	0.090	9.5%	249	0.170	0.193	8.0%
Rule controller	0.99	0.085	0.9%	12	0.117	0.227	9.8%
DWA (privileged)	0.93	0.066	0.2%	4	0.087	0.172	2.4%
Tabular Q seed 3 ★	0.86	0.087	12.1%	204	0.134	0.196	12.8%
Go-to-goal	1.07	0.118	6.9%	14	0.096	0.320	42.1%

Path efficiency = straight-line distance / driven path for successful robots; it can exceed 1 because a robot stops within 0.12 m of its goal point.

- PPO drives fastest and passes closest to other robots, leaving conflicts to the safety layer. DQN keeps more distance, drives slower and stops more.
- Both learned policies switch steering direction 200–250 times a minute (jittery commands); the planners 4–12 times.
- The rule controller skims obstacles (tuned margin 2 cm; 117 near-miss steps per robot vs PPO's 1.2): harmless with exact 2D geometry, a likely reason it lost ground in Gazebo.

7 Cost: deployment, compute and energy

Method	Parameters	Model size	One robot's decision (median / p99, µs)
PPO / DQN (same network)	92,717	372 KB	113 / 83
Tabular Q (coarse / fine)	291,600 / 7.1 M	1.2 MB / 28 MB	144 / 84
Rule controller	7 numbers	—	160
Go-to-goal	—	—	22
DWA (privileged)	—	—	needs simulator state

All are far below the 100 ms control period. In Gazebo (four robots per call) the median decision took 98 µs (PPO), 84 µs (DQN), 1.2 ms (rule).

Training and evaluation compute

Work	Where	Wall time	Compute	Energy
Tabular Q: 13 runs (tuning, finals, fine)	laptop, 1 thread each	6.4 h summed	6.3 CPU h	~24 Wh (est.)
Rule tuning: 50 parameter sets	laptop, 2–7 processes	1.7 h	4.3 CPU h	~16 Wh (est.)
DQN tuning: 6 × 1M steps	Colab T4 + 2 vCPU	1.0 h summed	GPU 4–7% busy	29 Wh GPU (measured)
DQN finals: 3 × 6M steps, concurrent	Colab T4 + 2 vCPU	1.75 h each	~955 steps/s per run; GPU 8–15% busy	~50 Wh GPU (est.)
Gazebo, 20 layouts	laptop	18–31 min per method		
2D evaluation, 100 layouts	laptop	1–5 min per condition		

CPU energy is an **estimate** (CPU seconds × 3.75 W per busy thread): Intel's RAPL energy counters are root-only since CVE-2020-8694 and were deliberately left closed. GPU energy was measured (NVML). The T4 was mostly idle: a 93k-parameter network is tiny for it, and Colab's two CPU cores (simulator, replay buffer) were the bottleneck. PPO's original 8M-step training took about 2 h on the laptop; its energy was not measured.

8 Integrity: deviations, bugs, limitations

Protocol amendments

1. **Finer tabular Q** tried after the coarse table plateaued at ~60% with 99% of states visited; it lost on validation, so the coarse table stands. One exploratory 8M run is reported separately.
2. Rule grid **extended past its edge** within the pre-set 50-set budget, decided before any rule test result. The final choice still sits on the edge of two parameters.
3. **DQN trained 6M of 8M steps**: the Colab runtime was reset and the replay buffers were lost; the 6M checkpoints were kept by the owner's decision. This can only disadvantage DQN; two of three seeds were still improving.

Bugs found and fixed before any reported number

- Tabular Q reused the terminal state as the next state after a reset (0.6% of updates): first tuning sweep discarded.
- The energy meter double-counted guest CPU time.
- The unattended pipeline had a path and a shell-strictness bug: 70 minutes lost, no result affected.

Findings about the original project

- PPO's checkpoint was selected on layouts 0–99, so its 91.5% there is slightly optimistic (90.2% on clean layouts).
- The training layouts predate the goal-visibility rule: 385 of 2000 differ from today's generator. All learners here trained on exactly PPO's layouts, so the comparison is fair.

Limitations

- **Tuning effort favours PPO** (10 runs plus reward and environment fixes).
- PPO is one trained policy; no seed spread is available.
- Gazebo used 20 layouts: intervals of ±6–9 points.
- The 2D simulator's motion model is exact, which flatters planners that predict motion.
- CPU energy is estimated, not measured.

Everything is reproducible from the repository: protocol `benchmark/PROTOCOL.md`, lab notebook `benchmark/LOGBOOK.md`, tables `benchmark/RESULTS.md`; `benchmark/make_results.py` recomputes every statistic from the raw per-robot files and `benchmark/make_report.py` rebuilds this PDF.